Do Skills Make ML-Engineering Agents Measurably Better?

A System for Building, Discovering, and Evaluating Agent Skills

Kuntal Kokate

PengTao Lab, University of California, San Diego

kukokate@ucsd.edu

June 2026

Abstract

Agent Skills—structured Markdown playbooks injected into a language-model agent's context—are increasingly used to specialise general agents for narrow domains. Yet there is little evidence that a skill changes what an agent *builds*, as opposed to what it *says*. This report presents an end-to-end system that (i) **builds** a progressive-disclosure skill from a single package URL, (ii) **discovers** and safely installs existing community skills, and (iii) **evaluates** whether a skill makes a machine-learning-engineering (MLE) agent measurably better. The intended end state is a *library* of builder-made skills handed to the agent, from which it selects a few of its own choosing while solving a task; the evaluation framework is built to test precisely that. The evaluation is a two-stage pipeline: a fast, CI-style local test that asks whether a skill *fires* and surfaces the right facts, and a paired with-skill / without-skill A/B framework that runs a real MLE search agent (MLEvolve) on parameter-efficient fine-tuning (PEFT) tasks and grades the outcome with an *independent held-out grader*. We describe the architecture, the progressive-disclosure injection mechanism, the trustworthy-metric design, and the Kubernetes runtime, and we report preliminary SAMSum results from the held-out grader: in a four-cell 2×2 run, two cells scored validly (ROUGE-L 0.29 with-skill and 0.23 without-skill, on different seeds) while an output-contract bug invalidated the other two, so no paired lift can yet be claimed. We close with the honest limitations of these spike runs and the design of the full sweep.

1 Introduction

A general-purpose coding agent can be specialised by injecting an *Agent Skill*: a Markdown document describing when and how to use a particular library or technique, following a *progressive-disclosure* convention in which a short description is always loaded, a body is loaded on trigger, and reference files are loaded only as needed. Skills are cheap to write and easy to share, but their effect is usually asserted rather than measured. The central question of this project is deliberately operational:

> *Does an MLE agent build a measurably better pipeline* with *a skill than* without *it—and, if so,* where *in the pipeline does the help land?*

Answering this requires three capabilities, which map onto the three loosely-coupled bodies of work in this project: a way to **build** good skills, a way to **discover** existing ones, and—the research focus—a rigorous way to **evaluate** their effect on a downstream agent.

The intended end state. The three capabilities are not independent deliverables but stages of one loop. The builder produces many skills into a shared *library*; at task time the *entire* library—not a single hand-picked skill—is made available to the MLE agent, which selects a few skills of its own choosing as it works, loading only what each step needs. The evaluation framework exists to test exactly this arrangement: whether a builder-made library with agent-driven selection makes the agent measurably better, and whether the agent selects the *right* skills.

The remainder of this report describes each capability in turn, with the bulk of the discussion devoted to the evaluation framework.

2 System Overview: Three Agents

The build and discovery capabilities are implemented as two OpenClaw agents; a third agent supports the local evaluation stage. Table 1 summarises them.

Table 1: The three agents and their roles.

Agent	Role	Status
`ai-skill-builder`	Synthesises a `SKILL.md` (plus references and scripts) from one documentation URL.	Phase 1.5, v1.4.0
`ai-skill-scout`	Searches GitHub for existing skills, ranks by trust, and installs them after a security scan.	Phase 1, complete
`skill-tester`	Drives the Stage-1 local evaluation; an MCP sidecar captures every prompt for measurement.	In production

2.1 Skill-Builder

The builder turns one URL into one skill. Its pipeline is: RESOLVE the owner/repo from the URL; FETCH documentation, README, and examples (optionally closed bug issues and Stack Exchange Q&A); EXTRACT hardware hints with a regex side-channel; SCAN community text for indirect-prompt-injection (IPI); PLAN the file structure with one LLM call; SYNTHESISE the body, reference files, templates, and evals in parallel; VALIDATE (a 60-pattern security scanner across seven threat categories, a line cap, a shell-command-fabrication check, and `py_compile` on templates); run a TRIGGERING EVAL that judges the description against decoys and rewrites it once if it loses; ASSEMBLE the YAML frontmatter *deterministically in Python*; and WRITE the bundle to the workspace.

Two design choices matter. First, **the model never writes the frontmatter**: the `name`, install steps, runtime MCP declarations, and a content hash for provenance are assembled by code, removing a class of hallucination. Second, **forum text is excluded by design**; Stack Exchange (licensed, vote-deduplicated, attribution-preserving) and closed issues are the community substrate, because unmoderated forums carry disproportionate prompt-injection risk. On the canonical `peft-tuning` skill the builder's triggering test scored an F1 of 1.000 (10 should-fire and 10 near-miss prompts, 60 judge calls).

2.2 Skill-Scout

The scout's source is GitHub only. It expands the user's query into synonyms via an LLM, merges `gh search` results, fetches each candidate's full `SKILL.md`, and scores *trust* (known orgs / star count), *completeness* (0–10), and a *classification* (ADOPT / EXTEND / GAP). Candidates are ranked by `(trust, class, completeness, stars)` so a high-trust skill that needs extending outranks a polished one of unknown origin. Crucially, nothing reaches the workspace until it has

been quarantined to `/tmp` and cleared the *same* 60-pattern scanner the builder uses—one source of truth for what is dangerous—and the user has approved the candidate.

2.3 Skill-Tester

The tester agent supports Stage 1 of the evaluation (Section 3.1). An MCP sidecar captures native tool calls, a bash side-channel log, and narration so that triggering and functional lift can be measured against a baseline agent, and reply text is preserved for offline re-grading.

3 Evaluation Methodology

The evaluation is a two-stage pipeline. Stage 1 asks a cheap question in isolation; Stage 2 asks the expensive question inside a full agent run. The division lets us reject weak skills before spending GPU hours.

3.1 Stage 1: Local Skill Evaluation

Stage 1 runs on every build (CI-style, $\approx$ 5 minutes, $\approx$ \$1) and follows the Anthropic `skill-creator` protocol. It reports four metrics: a **triggering F1** over a 20-prompt 60/40 should-fire / near-miss split (target ≥ 0.85); a **functional pass-rate** on prompts with deterministic `must_contain` assertions, run with and without the skill; a **lift** (the difference between the two); a **citation rate** (does the reply name a reference file; target $\geq 80\%$); and a **token-cost ratio** (target $\leq 2\times$). A known limitation is *saturation*: on canonical questions the base model already knows the answer, so deterministic checks pass with or without the skill, and the lift appears only on harder, version-specific questions—the very effect Stage 2 is designed to surface.

3.2 Stage 2: The MLEvolve A/B Framework

Stage 2 is the core contribution. It runs a real MLE agent on a freeform task twice—once with the skill injected and once without—holding everything else fixed, and measures the difference.

Agent. The agent is **MLEvolve-generic**, a Monte-Carlo graph search that generates, executes, and evolves candidate code nodes (a fork of AutoMLGen's search, pinned to a fixed commit), driven by `deepseek/deepseek-v4-pro` via OpenRouter. It replaced an earlier AIDE-based agent because its *subprocess-per-node* execution avoids a fork-after-CUDA failure mode that crashed the GPU pod.

Experimental design. Every controllable variable is held constant across the two cells: the same backbone (`Qwen/Qwen2.5-3B-Instruct`), the same task, the same random seed, and the same skill *library*. The *only* difference is whether that library is made available to the agent; when it is, the agent selects from it, and an empty library reproduces the baseline exactly. The current single-seed spike runs held one skill (`peft-tuning`) fixed—a "skill vs. no skill" contrast—whereas the designed end state is a multi-skill library from which the agent chooses, which additionally exposes a *selection-quality* signal (described below). The task pool is three contract-only PEFT fine-tuning tasks (Table 2); the full sweep is 3 seeds $\times$ 3 tasks $\times$ 2 cells = 18 trajectories, with a 95% confidence interval estimated from paired-seed variance.

Skill injection by progressive disclosure. MLEvolve performs single-shot code generation and cannot open a file to read a skill on its own. The injector stands in for that, mapping Anthropic's Discovery $\rightarrow$ Activation $\rightarrow$ Execution model onto the single seam (`get_impl_guideline_from_agent`) that every code-generation agent calls, via a `sys.meta_path` import hook that patches all four agents (draft, improve, debug, evolution):

Table 2: Stage-2 task pool.

Task	Domain	Metric
`samsum`	Dialogue summarisation (SFT)	ROUGE-L F1
`gsm8k`	Math reasoning (SFT)	Exact-match accuracy
`boolq`	Yes/no question answering (SFT)	Accuracy (test labels withheld)

- **Tier 0 — Discovery (every node).** A catalogue—each skill's name, one-line description, and reference filenames ($\approx$ 150 tokens for the whole library)—is appended to the implementation guideline, so the agent is always aware of what exists.
- **Tier 1 — Activation (per node).** A temperature-0 selector runs once per node, keyed to the current stage, and decides which skill(s) this step actually needs.
- **Tier 2 — Execution.** The same selector chooses which `references/*.md` files to splice in—never the whole library, never every body into every node.

When the skill library is empty (the without-skill cell), there is no catalogue and no selector call, so the guideline passes through byte-for-byte unchanged—guaranteeing the two cells differ only in the treatment.

Trustworthy metric. An agent that reports its own score will learn to report a good one. Therefore the headline number never comes from the agent. After the run exits, an *independent held-out grader* (`mleval.grader`) recomputes the task metric from the agent's preserved `submission.csv` against references the agent was never shown. MLEvolve's own self-reported validation score is retained only as the tree-search signal and a drift diagnostic; in one run a self-reported 0.81 graded out as invalid, which is exactly the gaming this design prevents.

Measurement layers. Beyond the L1 outcome (held-out grader + paired Lift), an **L2** layer attributes *where* the skill helped using a 6×16 pipeline-stage taxonomy and three co-location-proof metrics (clean-reach, rework, failure-modes) derived from an AST classifier over the agent's code; an **L3** layer records wall-clock, token, and dollar cost so a skill that helps but slows the agent is not mistaken for a free win. Finally, because the injector logs every `select_skills` call, a **selection-quality** layer measures whether the agent chose the right skills—the precision and recall of its per-task selections against a hand-labelled oracle of which library skills are genuinely relevant to each task. This layer is meaningful only once the library holds more than one skill, and becomes the headline as the project moves from the single-skill contrast to the full library.

4 Implementation and Runtime

The framework runs on the UCSD Nautilus NRP Kubernetes cluster: one container image per agent plugin, and one Pod per `(task` $\times$ `cell` $\times$ `seed)` trajectory. The MLEvolve agent is adapted at runtime by a small set of import-time monkey-patches (a "sidecar"), kept to the minimum needed for a fair, reproducible A/B:

- `seed` — pins `random` / `numpy` / `torch` RNG from the trajectory's seed, enabling paired-seed comparison;
- `prompt_logger` — captures every prompt, response, and token count;
- `token_budget` — raises the default output cap 16,384 $\rightarrow$ 32,768 to stop a mid-output truncation that corrupted generated code;
- `skill_injector` — the only patch that carries the treatment (Tier-0/1/2 above).

Separately, a build-time patch neutralises an upstream "Kaggle grandmaster" persona that was driving the agent off-task onto the wrong dataset. The container entrypoint handles signals so that the analyzer chain always runs and the trajectory's outputs always land on the shared PVC, even on a deadline kill.

5 Preliminary Results

The harness has been validated end-to-end across a series of SAMSum spike runs; the most complete is `spike-018`, which ran the full 2×2 (with/without skill $\times$ seeds 0–1) under the independent held-out grader. Of the four cells, only two produced a submission the grader could score—an output-contract bug (a submission with the wrong columns) invalidated one cell in *each* arm—so there is no seed-matched pair and **no Lift can yet be computed**. The two valid held-out ROUGE-L scores are 0.288 (with-skill, seed 1) and 0.227 (without-skill, seed 0); coming from different seeds, they are indicative only. The grader also rejected two inflated self-reported scores (0.94 and 0.81) as invalid—precisely the off-task drift it was built to catch (an earlier `spike-012` smoke had reported 0.4331 on a self-reported metric whose control had drifted to a different task). On *process* quality the with-skill arm is consistently stronger: a self-correction rate of $\approx$0.6 versus $\approx$0.15, markedly less redundant work (3–4 versus 9–12 redundant nodes), and a significant pipeline-stage redistribution ($\chi^2 = 65.1$, $p \approx 3 \times 10^{-9}$).

gsm8k. The `gsm8k` task was rewritten to an MLE-Bench-aligned held-out re-split (labels withheld on the test set; a carved validation slice for the agent's own signal). Run `spike-023` then validated the held-out pipeline end-to-end: a without-skill node produced a valid 1,319-row submission scoring **0.61 exact-match** (809/1,319) under the held-out grader (graded from the preserved submission; the graceful-kill path did not auto-persist the score file). Two findings shaped the next iteration. First, a skill-router regression left the treatment arm effectively empty: an $\approx$3 KB harness-rules header pushed the task's model, method, and data past the selector's 1,500-character cap, so the per-node selector returned no skills and no skill body ever reached a code-generation prompt; this was fixed with an explicit end-of-rules sentinel and a larger cap. Second, the binding constraint is *generation* throughput, not training: every node trains in $\approx$10–15 minutes, and timeouts occur in the 1,319-example decode—one node had to ratchet its batch size $4 \to 32$ and `max_new_tokens` $512 \to 200$ to finish within the 52-minute window. Because our harness imposes a hard 60-minute per-execution cap and marks any unfinished execution buggy (no partial credit), slowness is punished categorically; the planned response is to raise the cap rather than teach batching tricks in the instruction, which would erase the very signal the skill is meant to supply. No clean paired gsm8k A/B has completed yet.

These numbers should be read as *spike findings, not the verdict*. They come from a debugging-era sweep rather than the full paired 18-trajectory run, and the decisive failures hit *both* cells—the output-contract bug above (a wrong id column), fork-after-CUDA segmentation faults, and a period in which the skill router was blind to the task because the harness rules pushed the task description past the selector's character budget. Each was diagnosed and fixed; the harness now grades, retries, and harvests trajectories cleanly, so a clean paired Lift awaits a rerun.

6 Issues Surfaced in the Skill Builder

Running the skills inside a live agent—rather than only triggering them in isolation—exposed limitations in the builder that a triggering test cannot see. The clearest case is the `vllm-inference` skill on `gsm8k`: it passed the builder's triggering eval (F1 ≥ 0.85; the live selector picked it on 10 of 11 nodes) yet made no positive behavioural difference. Three structural gaps explain the distance between "fires" and "helps":

- **Wrong tool, correctly avoided.** The task prescribes Hugging Face `AutoModelForCausalLM` with batched generation; vLLM is never intended. The agent used Hugging Face in *both* arms, so the earlier reading that "the agent ignored the vLLM skill" was a mis-diagnosis, not a skill defect.
- **Genre self-exclusion.** The skill's *NOT-for* section directs supervised and LoRA fine-tuning to `transformers`; because `gsm8k` is a LoRA task, the skill scoped *itself* out and steered the agent away—counterproductive rather than merely inert—despite its own decision tree listing offline evaluation as in-scope.
- **The decisive failure was functional, not selection.** Both arms ran Hugging Face regardless of what was selected; the runs failed on generation *hygiene*—`use_cache` disabled by gradient checkpointing, loose `max_new_tokens`, and validation generation every epoch—which the co-resident `peft-tuning` skill (an older `1.3.0` build) documents as API surface, not as workflow choreography.

The triggering eval missed this because its judge competes the new skill against five canned generic decoys rather than its real co-resident siblings, so the disambiguation that mattered—fine-tune-and-evaluate should lead with `peft-tuning`, not `vllm-inference`—was never tested, and because the judge scores short user messages whereas the live selector reads a multi-paragraph task instruction. Two creator-level fixes follow: (1) a *library-aware* triggering eval that judges a skill against its actual siblings, including hard fine-tune-versus-serve cases; and (2) a *functional* (choreography) eval that rewards a skill for encoding operational gotchas, not merely for owning the right keywords. Tightening descriptions alone would not have caught this.

7 Discussion and Limitations

The clearest lesson is methodological: a credible skill-effect number requires a grader the agent cannot reach, and a treatment whose only difference from the baseline is the skill itself. A second lesson, consistent with the literature, is that the skill's **description** is its single highest-leverage component—wording alone swings invocation by more than an order of magnitude with capability held fixed—so a skill that never fires zeroes out every other quality it has. The `vllm-inference` case (preceding section) sharpens this in the other direction: discovery is *necessary but not sufficient*—a skill can be selected on nearly every node and still fail to help—so a credible evaluation must reach past triggering into functional behaviour. Limitations remain: the AST stage classifier is a rule-table MVP pending a call-graph upgrade; the LLM judge for subjective assertions is off by default; and the full paired sweep, with confidence intervals, has not yet been run.

8 Conclusion and Future Work

This project delivers a closed loop for Agent Skills—build, discover, and evaluate—with the evaluation framework as the research contribution. The immediate next step is the full paired A/B across `samsum`, `gsm8k`, and `boolq`, reporting mean Lift with a 95% confidence interval, the L2 stage-level attribution, and—once the library holds several builder-made skills—the agent's skill-selection precision and recall against a per-task oracle. The promotion rule is deliberately conservative: a skill is shipped only when it lifts *both* task outcome and recipe fidelity, never one at the expense of the other. In parallel, the builder's own evaluation will be hardened along the two axes the `vllm-inference` case exposed—a library-aware triggering eval and a functional choreography check—so that a skill which *fires* reliably also *helps*.